

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroller

Christopher Joenathan Hindarto - 5024241028

07 November 2026

1 Langkah-Langkah Percobaan

Pada praktikum Modul 1, dilakukan lima percobaan yang mencakup pengenalan board STM32 Blackpill dan ESP32, konfigurasi pull-up/pull-down resistor, debouncing, serta pembacaan ADC. Sebelum memulai, dilakukan persiapan lingkungan pengembangan yaitu instalasi Visual Studio Code beserta ekstensi PlatformIO, serta pembuatan satu project PlatformIO yang digunakan untuk seluruh percobaan pada modul ini. Setiap percobaan cukup mengganti isi `platformio.ini` dan file sumber (`src/main.cpp` atau `src/main.c`) sesuai kebutuhan.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Tujuan: Memahami arsitektur board STM32 Blackpill dan melakukan flashing program Blink menggunakan ST-Link dengan framework Arduino.

Langkah Kerja:

1. Memastikan project Modul 1 sudah dibuat pada PlatformIO dan mengisi `platformio.ini` dengan konfigurasi berikut:

Listing 1: Konfigurasi platformio.ini untuk STM32 Blackpill

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink

```

2. Menghubungkan ST-Link ke Blackpill menggunakan kabel jumper female-female dengan konfigurasi pin:

   - SWDIO ↔ SWDIO (jalur data/debug)
   - SWCLK ↔ SWCLK (jalur clock/debug)
   - GND ↔ GND (referensi ground)
   - 3.3V ↔ 3V3 (tegangan target board)

   Kemudian mencolokkan ST-Link ke komputer lewat USB.

3. Menulis kode program Blink pada file `src/main.cpp`:

Listing 2: Kode Program Blink STM32 Arduino

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

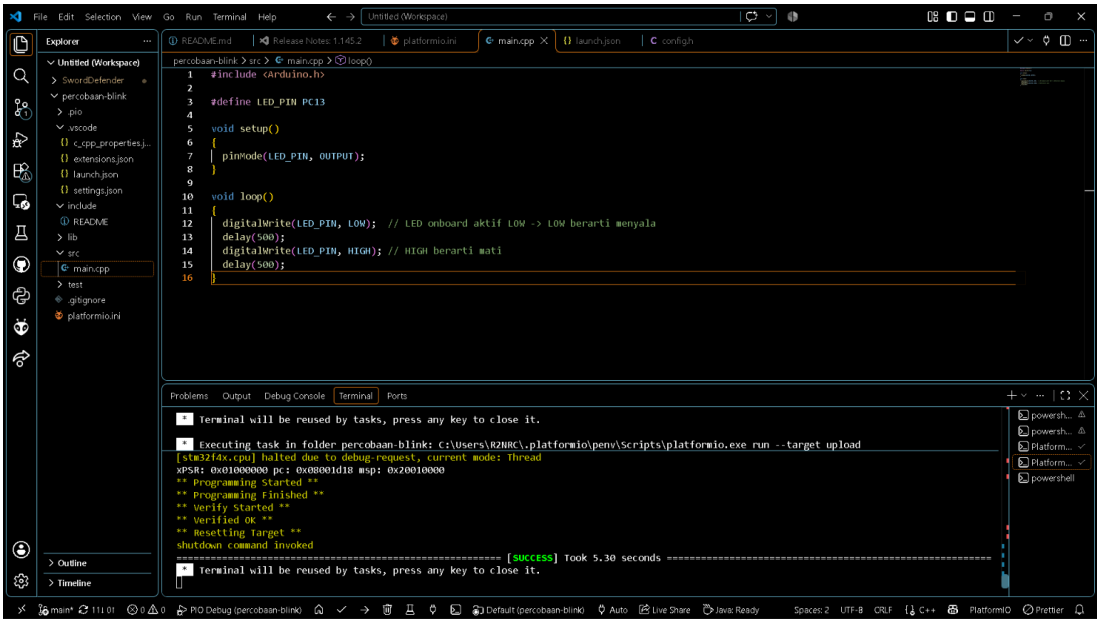
void loop()
```

```
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}

```

4. Melakukan **Build** dan **Upload** melalui PlatformIO.

5. Mengamati LED onboard pada pin PC13: nyala selama 500ms, mati selama 500ms, berulang terus-menerus.



Gambar 1: Hasil Percobaan 1 – Program Blink STM32 Blackpill berhasil di-upload melalui ST-Link dan berjalan dengan status `[SUCCESS]`

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Tujuan: Memahami arsitektur ESP32 dan struktur program berbasis ESP-IDF, serta berhasil menjalankan program Blink menggunakan framework tersebut.

Langkah Kerja:

1. Mengganti isi `platformio.ini` dengan konfigurasi ESP32 dan framework ESP-IDF:

Listing 3: Konfigurasi platformio.ini untuk ESP32 ESP-IDF

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf

```

   Menunggu proses *refresh*/re-index konfigurasi PlatformIO hingga selesai sebelum melanjutkan.

2. Mencolokkan ESP32 DevKit ke komputer lewat USB dan memastikan port terdeteksi.

3. Menghapus file `src/main.cpp` dan membuat file baru `src/main.c`, kemudian menulis kode program Blink ESP-IDF:

Listing 4: Kode Program Blink ESP32 ESP-IDF

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

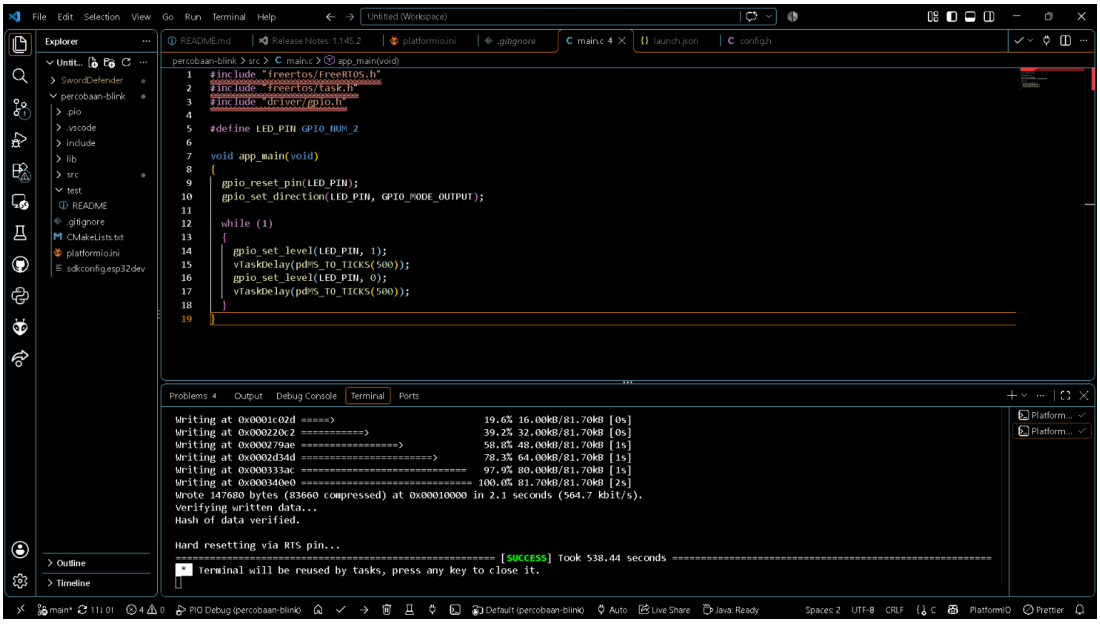
void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}

```

4. Melakukan **Build** dan **Upload**.

5. Mengamati LED onboard ESP32 (GPIO 2) berkedip dengan interval 500ms.



Gambar 2: Hasil Percobaan 2 – Program Blink ESP32 dengan framework ESP-IDF berhasil di-upload dan berjalan dengan status `[SUCCESS]`

1.3 Percobaan 3: Pull-up dan Pull-down – Eksternal vs Internal (ESP32 + Framework Arduino)

Tujuan: Memahami, mengimplementasikan, dan membandingkan pembacaan tombol menggunakan resistor pull-up/pull-down yang dipasang secara eksternal maupun diaktifkan secara internal pada ESP32 dengan framework Arduino.

Langkah Kerja:

1. Mengganti isi `platformio.ini` ke konfigurasi Arduino:

Listing 5: Konfigurasi platformio.ini untuk ESP32 Arduino

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino

```

   Menghapus `src/main.c` dan membuat kembali file `src/main.cpp`.

2. **Bagian A – Pull-up/Pull-down Eksternal:** Memasang dua pushbutton di GPIO 32 (pull-up eksternal dengan resistor 10kΩ ke 3.3V) dan GPIO 33 (pull-down eksternal dengan resistor 10kΩ ke GND).

3. Menulis Kode Program A (Eksternal) pada `src/main.cpp`:

Listing 6: Kode Program A – Pull-up & Pull-down Eksternal

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}

```

4. Melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor.

5. **Bagian B – Pull-up/Pull-down Internal:** Melepas kedua resistor eksternal (tombol tetap terpasang), mengubah koneksi pushbutton 2 dari 3.3V ke GND sesuai skema internal.

6. Mengganti isi `src/main.cpp` dengan Kode Program B (Internal):

Listing 7: Kode Program B – Pull-up & Pull-down Internal

```cpp
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up internal
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
}

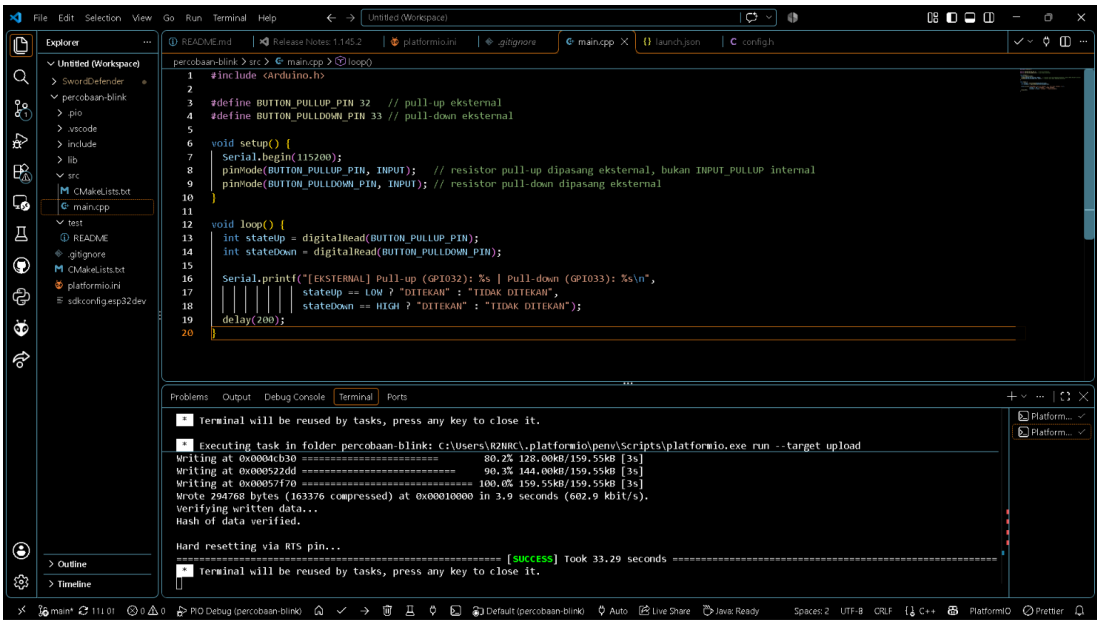
void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}

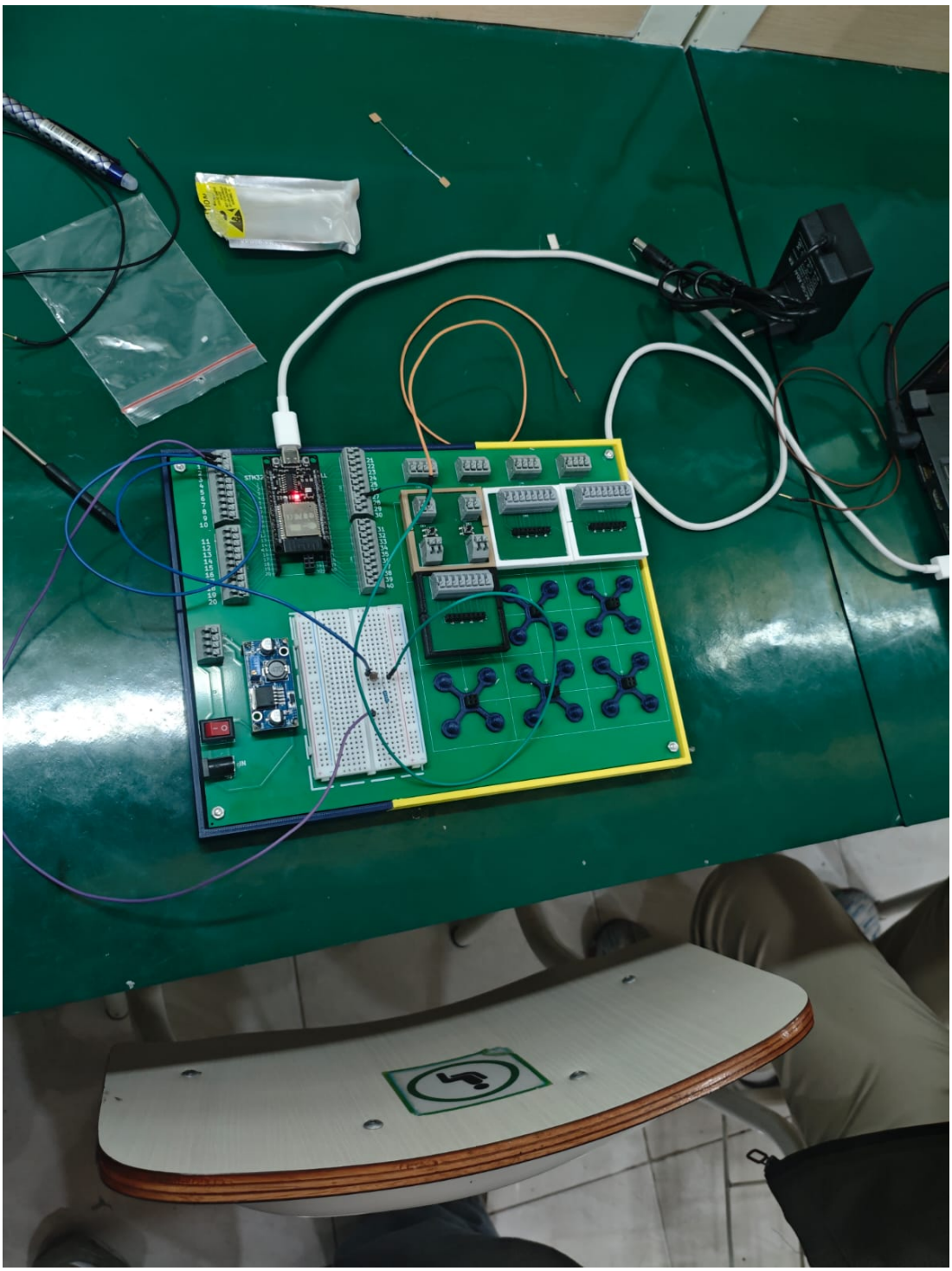
```

7. Melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor.

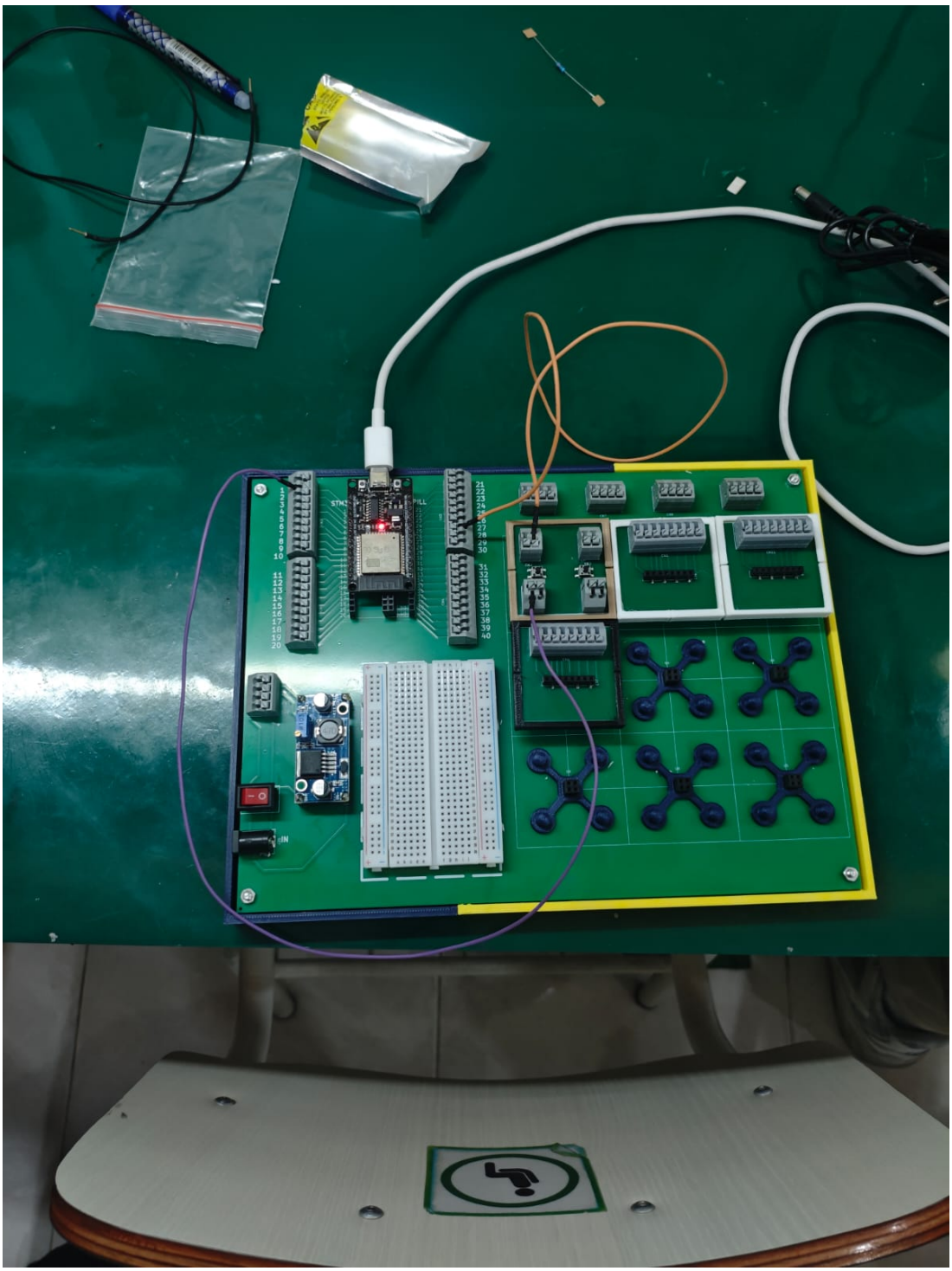
8. Membandingkan hasil Bagian A dan Bagian B. Hasil yang diperoleh seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down sebaliknya.



Gambar 3: Hasil Percobaan 3 Bagian A – Kode Program Pull-up & Pull-down Eksternal berhasil di-upload dengan status `[SUCCESS]`



Gambar 4



Gambar 5

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Tujuan: Mengidentifikasi permasalahan bouncing pada tombol mekanik dan mengimplementasikan debouncing berbasis software pada ESP32 dengan framework Arduino.

Langkah Kerja:

1. Memasang satu pushbutton pada GPIO 25 ESP32 dengan konfigurasi pull-up internal (satu kaki ke GPIO, kaki lain ke GND).

2. Konfigurasi `platformio.ini` tetap sama seperti Percobaan 3 (ESP32 Arduino), sehingga tidak perlu diubah.

3. Menulis kode program perbandingan counter tanpa debouncing dan dengan debouncing pada `src/main.cpp`:

Listing 8: Kode Program Debouncing

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int reading = digitalRead(BUTTON_PIN);

  // ===== Tanpa Debouncing =====
  if (reading == LOW && rawLastState == HIGH) {
    counterNoDebounce++;
  }
  rawLastState = reading;

  // ===== Dengan Debouncing =====
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai =====
  static unsigned long lastPrintedNoDebounce = 0;
  static unsigned long lastPrintedWithDebounce = 0;
  if (counterNoDebounce != lastPrintedNoDebounce ||
```

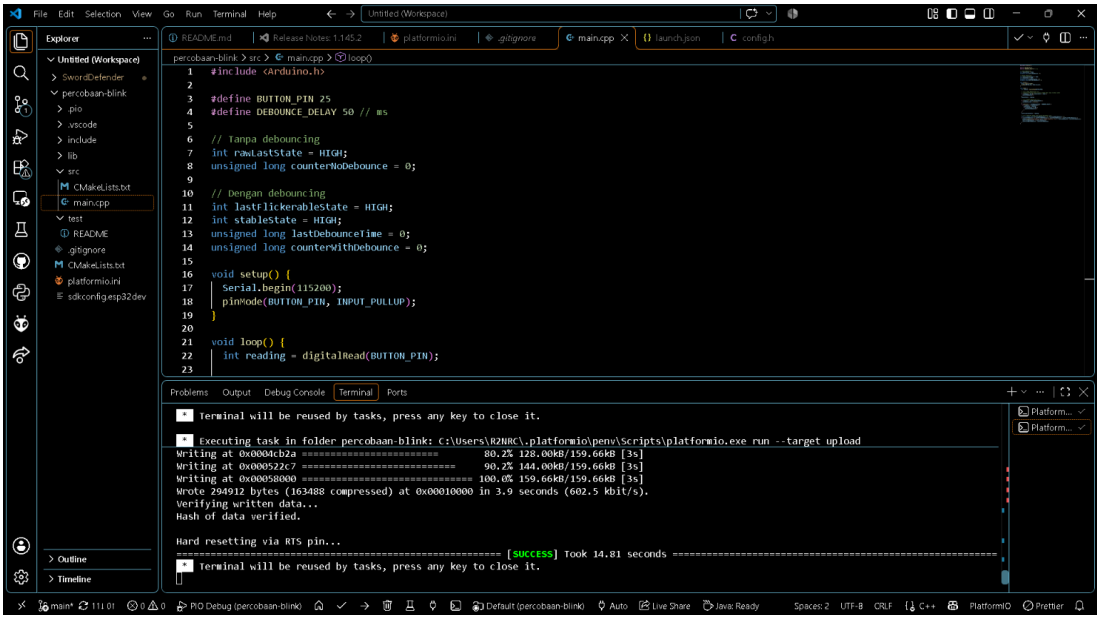
```
      counterWithDebounce != lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n",
                  counterNoDebounce, counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}

```

4. Melakukan Build dan Upload, kemudian menekan tombol satu kali dan membandingkan nilai kedua counter di Serial Monitor.

5. Mengulangi penekanan dengan cara berbeda (cepat dan lambat) untuk mengamati efek bouncing. Counter tanpa debouncing dapat naik lebih dari 1 untuk satu tekanan, sedangkan counter dengan debouncing naik tepat 1.

6. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian 200 ms, lalu melakukan Build dan Upload untuk masing-masing nilai dan membandingkan ketiga hasil tersebut.



Gambar 6: Hasil Percobaan 4 – Kode program debouncing berhasil di-upload dengan status `[SUCCESS]`, terlihat variabel-variabel counter dan debounce pada kode

1.5 Percobaan 5: Akses GPIO Analog – Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Tujuan: Memahami ADC sebagai mode akses GPIO ketiga (analog input) dan mengimplementasikan pembacaan nilai analog sederhana menggunakan LDR.

Langkah Kerja:

1. Memasang LDR bersama resistor 10kΩ dalam konfigurasi pembagi tegangan: satu kaki LDR ke 3.3V, kaki lain ke GPIO 34 dan ke GND melalui resistor 10kΩ.

2. Konfigurasi `platformio.ini` tetap sama (ESP32 Arduino), ditambahkan `monitor_speed = 115200` untuk Serial Monitor.

3. Menulis kode program pembacaan ADC pada `src/main.cpp`:

Listing 9: Kode Program Pembacaan ADC dengan LDR

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

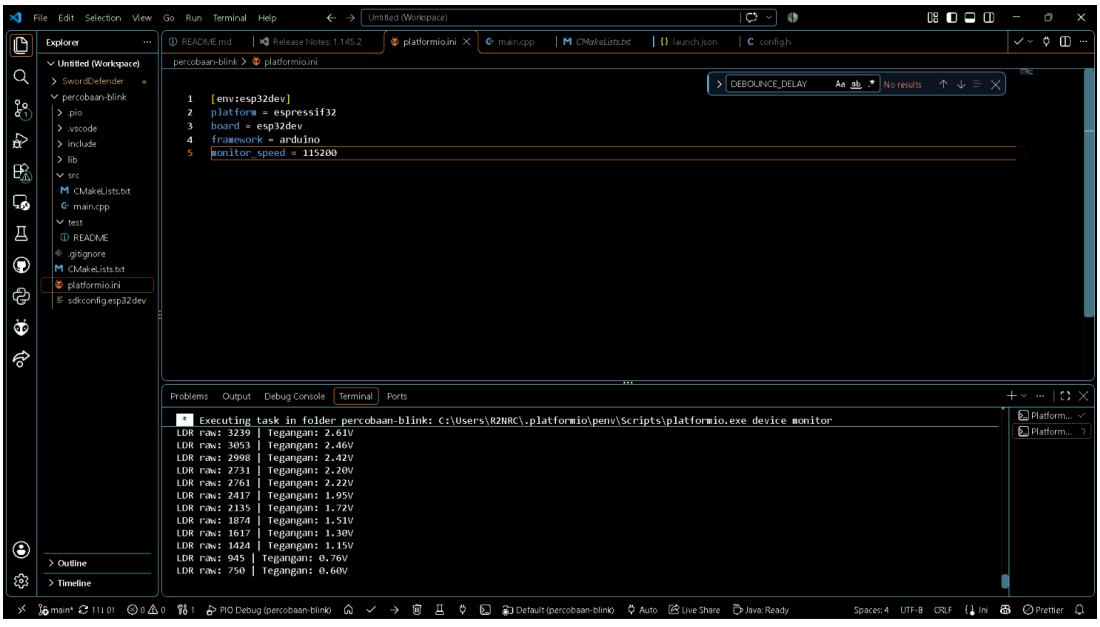
void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}

```

4. Melakukan Build dan Upload, kemudian membuka Serial Monitor.

5. Menutup LDR dengan tangan (nilai ADC menurun, tegangan turun) dan menyinari dengan lampu (nilai ADC naik, tegangan naik). Nilai mentah berubah dalam rentang 0–4095.



Gambar 7: Hasil Percobaan 5 – Pembacaan ADC dengan LDR pada Serial Monitor menunjukkan nilai raw dan tegangan yang berubah sesuai intensitas cahaya

2 Analisis Hasil Percobaan

2.1 Analisis Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Percobaan 1 berhasil dilaksanakan dengan baik. Program Blink pada STM32 Blackpill berjalan sesuai harapan, yaitu LED onboard pada pin PC13 berkedip dengan interval 500ms. Hal ini membuktikan

bahwa koneksi ST-Link ke Blackpill melalui protokol SWD (SWDIO, SWCLK, GND, 3.3V) telah dikonfigurasi dengan benar. Proses flashing berlangsung lancar dengan waktu sekitar 5,30 detik dan menampilkan status `[SUCCESS]`. Secara teori, LED onboard Blackpill bersifat *active LOW*, sehingga memberikan nilai LOW pada `digitalWrite()` akan menyalakan LED, dan ini terkonfirmasi pada praktikum. Tidak terdapat kendala berarti pada percobaan ini.

2.2 Analisis Percobaan 2: Pengenalan ESP32 dengan ESP-IDF (Blink)

Percobaan 2 juga berhasil dilakukan. Program Blink pada ESP32 menggunakan framework ESP-IDF berjalan dengan benar, LED pada GPIO 2 berkedip dengan interval 500ms. Perbedaan utama yang diamati antara framework Arduino dan ESP-IDF adalah pada struktur program: ESP-IDF menggunakan `app_main()` sebagai titik masuk program dan `vTaskDelay(pdMS_TO_TICKS())` untuk delay berbasis FreeRTOS, sedangkan Arduino menggunakan `setup()` dan `loop()` dengan `delay()`. Proses upload memakan waktu lebih lama (538,44 detik) dibandingkan Percobaan 1 karena firmware ESP-IDF berukuran lebih besar (147.680 bytes) akibat menyertakan komponen FreeRTOS dan driver native. Pada awal percobaan ini sempat mengalami sedikit *trouble* karena lupa mengganti konfigurasi `platformio.ini` dari konfigurasi STM32 ke ESP32, sehingga proses Build gagal. Setelah konfigurasi diperbaiki, percobaan berjalan lancar.

2.3 Analisis Percobaan 3: Pull-up dan Pull-down – Eksternal vs Internal

Kedua bagian percobaan (Bagian A dengan resistor eksternal dan Bagian B dengan pull-up/pull-down internal ESP32) menunjukkan hasil yang konsisten dan sesuai teori. Pada konfigurasi pull-up (GPIO 32), pin membaca HIGH saat tombol tidak ditekan (pin ditarik ke 3.3V melalui resistor) dan LOW saat tombol ditekan (pin terhubung ke GND). Sebaliknya, pada konfigurasi pull-down (GPIO 33), pin membaca LOW saat tidak ditekan dan HIGH saat ditekan. Hasil Bagian A dan Bagian B identik, membuktikan bahwa resistor pull-up/pull-down internal ESP32 yang diaktifkan melalui `INPUT_PULLUP` dan `INPUT_PULLDOWN` memiliki fungsi yang setara dengan resistor eksternal. Hal ini sesuai dengan dasar teori bahwa resistor internal berfungsi untuk mencegah kondisi *floating* pada pin GPIO. Tidak ada kesalahan berarti pada percobaan ini.

2.4 Analisis Percobaan 4: Debouncing pada Input Tombol

Percobaan debouncing menunjukkan hasil yang sangat jelas dalam mendemonstrasikan fenomena *bouncing* pada tombol mekanik. Ketika tombol ditekan satu kali, counter tanpa debouncing (`counterNoDebounce`) sering kali bertambah lebih dari 1 (misalnya naik 2–4 sekaligus), sedangkan counter dengan debouncing (`counterWithDebounce`) konsisten bertambah tepat 1. Hal ini sesuai teori bahwa kontak mekanik menghasilkan beberapa transisi HIGH-LOW dalam waktu sangat singkat (beberapa milidetik) akibat getaran fisik saat tombol ditekan/dilepas.

Pengujian dengan variasi nilai `DEBOUNCE_DELAY` menunjukkan *trade-off* antara keandalan debouncing dan responsivitas:

- **10 ms:** Counter debounced kadang masih menangkap bouncing (terlalu sensitif), tetapi responsivitas tombol sangat cepat.
- **50 ms:** Nilai optimal – counter debounced selalu akurat dan responsivitas masih baik.

- **200 ms:** Counter debounced sangat bersih dari bouncing, namun tombol terasa lambat merespons saat ditekan berulang dengan cepat karena penekanan yang terlalu cepat diabaikan.

2.5 Analisis Percobaan 5: Pembacaan ADC dengan LDR

Percobaan ADC berhasil mendemonstrasikan mode akses GPIO ketiga, yaitu analog input. Hasil pembacaan menunjukkan bahwa saat LDR terkena cahaya terang, resistansi LDR menurun sehingga tegangan pada titik tengah pembagi tegangan meningkat, dan nilai ADC mendekati 4095 (tegangan mendekati 3.3V). Sebaliknya, saat LDR ditutup tangan (gelap), resistansi LDR meningkat sehingga tegangan turun dan nilai ADC mendekati 0. Dari screenshot hasil percobaan, terlihat nilai raw berubah dari sekitar 3239 (2.61V) hingga 750 (0.60V) saat intensitas cahaya bervariasi. Hal ini membuktikan bahwa ADC mampu merepresentasikan kondisi analog secara kontinu (0–4095), berbeda dengan digital input yang hanya mengenal dua kondisi (HIGH/LOW). GPIO 34 yang digunakan merupakan pin input-only yang mendukung ADC1, sesuai rekomendasi untuk menghindari konflik dengan modul WiFi.

3 Hasil Tugas Modul

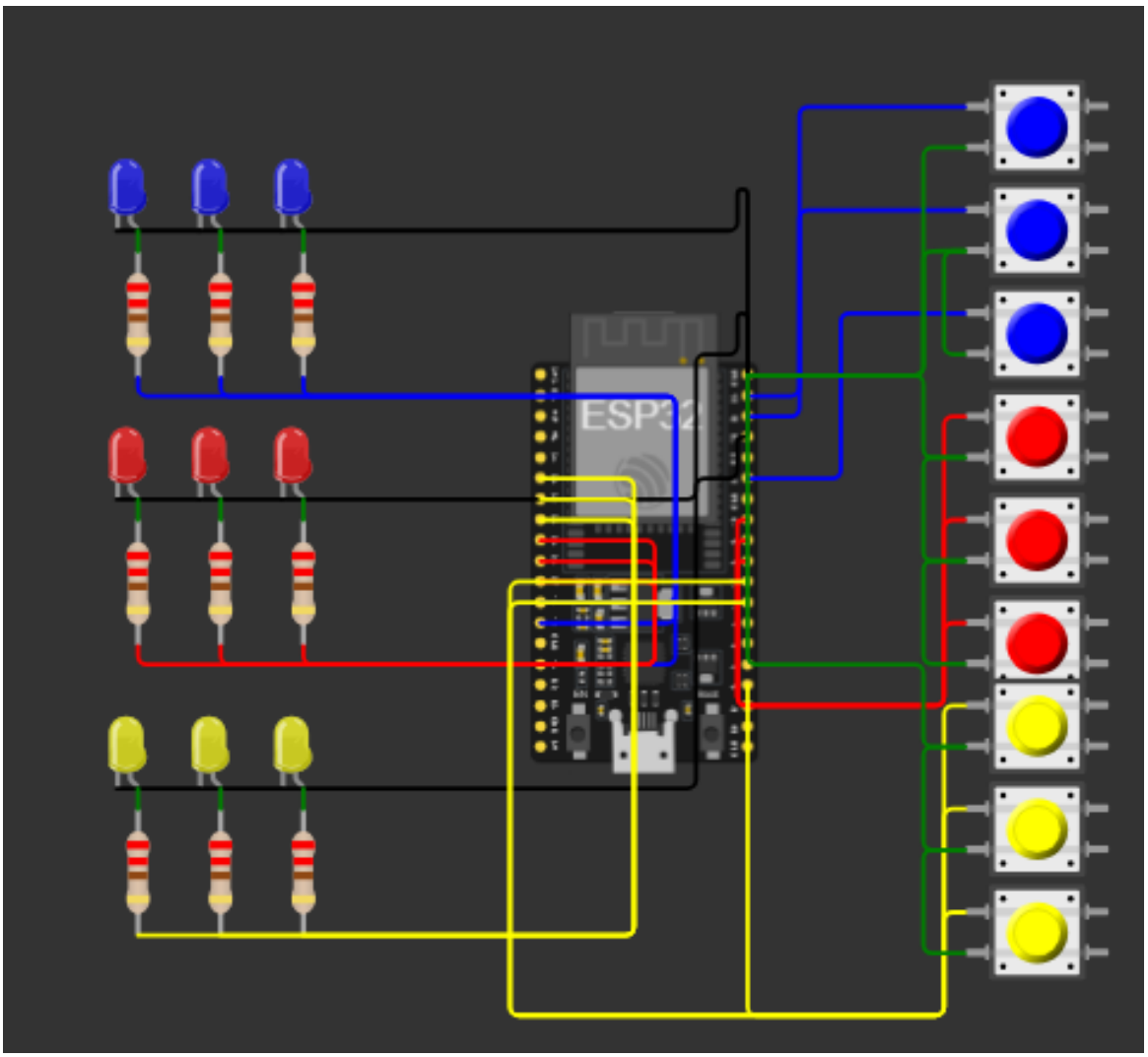
3.1 Simulasi Matrix Pushbutton 3×3 Mengendalikan Matrix LED 3×3

Simulasi ini menggunakan ESP32 dengan sembilan pushbutton dan sembilan LED yang disusun dalam bentuk matriks 3×3. Setiap pushbutton mengendalikan LED pada posisi yang sama. Ketika tombol ditekan, LED yang bersesuaian akan menyala, sedangkan ketika tombol dilepaskan, LED akan mati kembali.

Pada program ini diterapkan metode debouncing menggunakan fungsi `millis()` dengan waktu debounce sebesar 50 milidetik. Debouncing diterapkan secara independen pada setiap tombol agar perubahan kondisi tombol dapat dibaca dengan stabil dan tidak terdeteksi sebagai beberapa kali penekanan.

1. **Rangkaian Keseluruhan**

   Gambar berikut memperlihatkan rangkaian keseluruhan simulasi yang terdiri dari ESP32, matrix pushbutton 3×3, dan matrix LED 3×3 beserta koneksi antar komponennya.



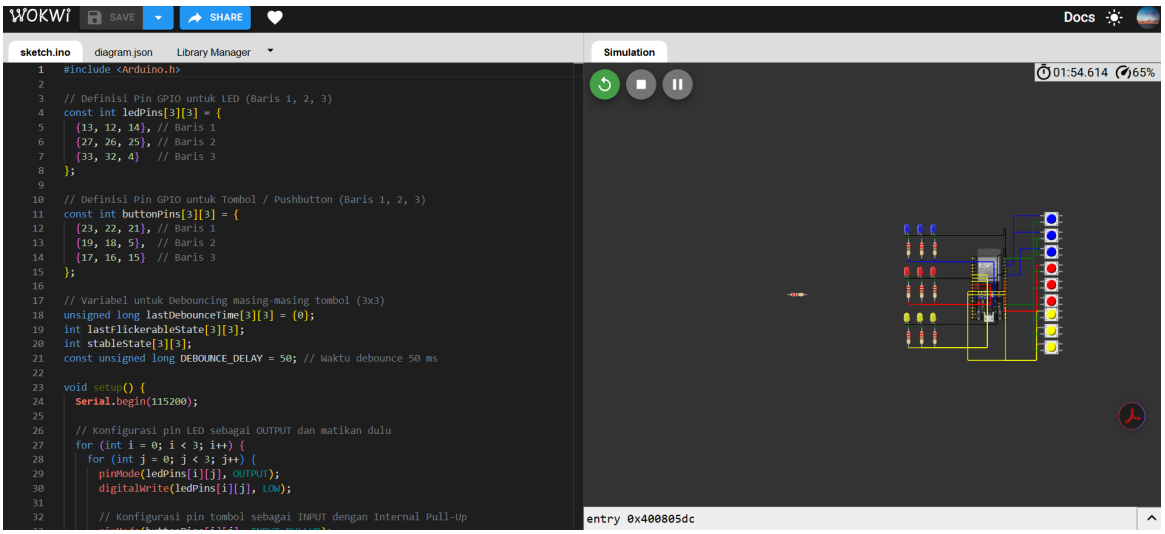
Gambar 8: Rangkaian keseluruhan Matrix Pushbutton 3×3 dan Matrix LED 3×3 pada Wokwi

2. **Pengujian Pushbutton dan LED**

   Pengujian dilakukan dengan menekan salah satu pushbutton. Sebagai contoh, ketika tombol pada baris pertama dan kolom pertama ditekan, LED pada posisi yang sama akan menyala. LED akan kembali mati ketika tombol dilepaskan.

3. **Bukti Program dan Penerapan Debouncing**

   Gambar berikut menunjukkan implementasi program pada ESP32 yang menerapkan debouncing menggunakan fungsi `millis()`. Serial Monitor digunakan untuk menampilkan perubahan status tombol setelah kondisi input dinyatakan stabil.



Gambar 9: Source code dan Serial Monitor penerapan debouncing pada matrix pushbutton

3.2 Jawaban Pertanyaan Analisis

3.2.1 Pertanyaan 1: Mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton?

Debouncing tetap diperlukan pada rangkaian matrix pushbutton 3×3 meskipun pada satu waktu biasanya hanya satu tombol yang ditekan, karena **bouncing adalah fenomena fisik pada kontak mekanik tombol itu sendiri, bukan masalah jumlah tombol**. Seperti yang dijelaskan pada bagian C.5 Dasar Teori, setiap kali sebuah tombol ditekan atau dilepas, kontak mekanik internal mengalami getaran (vibrasi) yang menghasilkan beberapa transisi sinyal HIGH-LOW dalam waktu sangat singkat (orde milidetik). Fenomena ini terjadi pada setiap tombol secara individual, terlepas dari apakah tombol tersebut berada dalam rangkaian tunggal maupun dalam matrix.

Pada rangkaian matrix 3×3, tanpa debouncing, satu kali penekanan tombol pada posisi tertentu dapat menghasilkan beberapa kali perubahan status LED yang berkedip-kedip cepat sebelum akhirnya stabil. Hal ini dapat mengganggu fungsionalitas sistem, terutama jika LED diatur dalam mode toggle (nyala/mati bergantian). Selain itu, dalam skenario di mana beberapa tombol ditekan hampir bersamaan, bouncing pada satu tombol dapat menyebabkan pembacaan yang tidak akurat pada proses scanning matrix, sehingga tombol yang tidak ditekan bisa saja terbaca aktif secara sesaat.

Oleh karena itu, setiap tombol dalam matrix harus memiliki mekanisme debouncing tersendiri dengan variabel waktu debouncing (`lastDebounceTime`) dan status stabil (`stableState`) yang independen, seperti yang diimplementasikan dalam program simulasi.

3.2.2 Pertanyaan 2: Analisis konfigurasi pull-up/pull-down pada pembacaan tombol matrix

Pada rancangan simulasi ini, setiap tombol dikonfigurasi menggunakan **pull-up internal** ESP32 melalui `pinMode(pin, INPUT_PULLUP)`. Dengan konfigurasi ini:

- **Saat tombol TIDAK ditekan:** Pin GPIO terbaca **HIGH** (3.3V), karena resistor pull-up internal menarik (“*pull*”) tegangan pin ke VCC.
- **Saat tombol DITEKAN:** Pin GPIO terbaca **LOW** (0V), karena tombol menghubungkan pin langsung ke GND, mengalahkan tarikan resistor pull-up.

Kondisi pin GPIO **tidak boleh dibiarkan *floating*** (tanpa pull-up maupun pull-down) karena jika kedua kaki tombol tidak terhubung ke referensi tegangan mana pun saat tombol tidak ditekan, pin GPIO berada dalam keadaan impedansi tinggi (*high-impedance state*). Dalam kondisi ini, pin sangat rentan terhadap noise elektromagnetik dari lingkungan sekitar, kabel yang berfungsi sebagai antena, atau crosstalk dari jalur sinyal lain pada breadboard. Akibatnya, nilai yang dibaca oleh `digitalRead()` menjadi **tidak pasti dan acak** – bisa terbaca HIGH atau LOW tanpa pola yang konsisten. Pada rangkaian matrix, kondisi floating ini sangat berbahaya karena dapat menyebabkan "phantom key press", yaitu sistem mendeteksi penekanan tombol padahal tidak ada tombol yang benar-benar ditekan, sehingga LED menyala secara tidak terkendali.

3.2.3 Pertanyaan 3: Perbedaan digital input vs analog input (ADC)

Jika salah satu tombol pada matrix digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), terdapat perbedaan mendasar antara pembacaan GPIO sebagai **digital input** dan sebagai **analog input**:

- **Digital Input (tombol):** Hanya mengenal dua kondisi diskrit, yaitu HIGH (logika 1, tegangan mendekati VCC) dan LOW (logika 0, tegangan mendekati GND). Mikrokontroler menggunakan *threshold* tegangan untuk menentukan apakah sinyal termasuk HIGH atau LOW. Semua variasi tegangan di atas threshold dianggap HIGH, dan di bawahnya dianggap LOW. Oleh karena itu, digital input hanya mampu merepresentasikan dua keadaan (ditekan/tidak ditekan).
- **Analog Input (ADC):** ADC (*Analog-to-Digital Converter*) mampu mengukur tegangan secara kontinu dan mengonversinya menjadi nilai digital. Pada ESP32 dengan resolusi 12-bit, ADC menghasilkan nilai dalam rentang 0–4095 yang merepresentasikan tegangan 0–3.3V. Artinya, ADC dapat membedakan **4096 level tegangan yang berbeda**, bukan hanya dua.

ADC dapat merepresentasikan lebih dari dua kondisi karena prinsip kerjanya yang menggunakan *successive approximation* atau metode konversi lainnya untuk mengukur besaran tegangan secara presisi, lalu mengkuantisasinya menjadi nilai biner multi-bit. Dengan LDR sebagai pembagi tegangan, perubahan intensitas cahaya menyebabkan perubahan resistansi LDR secara kontinu, yang menghasilkan variasi tegangan analog yang halus pada pin ADC. Misalnya, kondisi "gelap" bisa menghasilkan nilai 200, "redup" menghasilkan 1500, "terang" menghasilkan 3500 – semua merupakan kondisi yang valid dan dapat dibedakan, tidak mungkin direpresentasikan oleh digital input yang hanya mengenal HIGH/LOW.

4 Kesimpulan

Berdasarkan pelaksanaan praktikum Modul 1 – Pengantar Mikrokontroler dan Dasar Pemrograman, dapat disimpulkan bahwa:

1. Instalasi dan konfigurasi PlatformIO pada VSCode sebagai lingkungan pengembangan embedded berhasil dilakukan. PlatformIO memungkinkan penggunaan satu project untuk berbagai board dan framework dengan hanya mengubah konfigurasi pada file `platformio.ini`.
2. Board STM32 Blackpill (ARM Cortex-M4) dan ESP32 (Xtensa LX6 dual-core) memiliki arsitektur dan karakteristik yang berbeda. STM32 Blackpill memerlukan programmer eksternal (ST-Link)

untuk flashing, sedangkan ESP32 dapat langsung diprogram melalui kabel USB. Framework Arduino menyediakan abstraksi tingkat tinggi dengan `setup()`/`loop()`, sedangkan ESP-IDF menyediakan akses lebih native melalui `app_main()` dan API FreeRTOS.

3. Program Blink berhasil dijalankan pada kedua board, membuktikan bahwa konfigurasi hardware dan software telah benar. LED onboard STM32 (PC13) bersifat active LOW, sedangkan LED onboard ESP32 (GPIO 2) bersifat active HIGH.

4. Konfigurasi pull-up dan pull-down resistor, baik secara eksternal maupun internal, berhasil diimplementasikan dan menunjukkan hasil pembacaan tombol yang konsisten. Pull-up internal (`INPUT_PULLUP`) dan pull-down internal (`INPUT_PULLDOWN`) pada ESP32 dapat menggantikan resistor eksternal dengan fungsionalitas yang setara, sehingga menyederhanakan rangkaian.

5. Debouncing berbasis software menggunakan `millis()` efektif mengatasi fenomena bouncing pada tombol mekanik. Nilai `DEBOUNCE_DELAY` sebesar 50ms memberikan keseimbangan optimal antara keandalan pembacaan dan responsivitas tombol. Tanpa debouncing, satu penekanan tombol dapat terdeteksi sebagai beberapa event.

6. ADC sebagai mode akses GPIO ketiga (analog input) berhasil didemonstrasikan dengan LDR. Berbeda dengan digital input yang hanya mengenal dua kondisi (HIGH/LOW), ADC 12-bit pada ESP32 mampu mengonversi tegangan analog menjadi 4096 level digital (0–4095), memungkinkan pembacaan sensor yang memiliki output analog secara kontinu.

7. Seluruh hasil percobaan sesuai dengan teori yang dipelajari. Kendala minor yang dialami hanya berupa kelalaian dalam mengganti konfigurasi `platformio.ini` saat berpindah antar percobaan, yang dapat diatasi dengan lebih teliti memperhatikan konfigurasi sebelum melakukan Build.

5 Lampiran

5.1 Dokumentasi Saat Praktikum

Dokumentasi berikut merupakan bukti pelaksanaan percobaan yang telah dilakukan selama praktikum.



Gambar 10: Dokumentasi pelaksanaan percobaan saat praktikum